

Plataforma de recomendación inteligente para compras de supermercado

L. Lafaurie, A. Henao y B. Gutiérrez – Filiación Universidad Del Norte – Kilometro 5 Vía Puerto Colombia

I. INTRODUCCIÓN

La creciente digitalización del comercio ha generado una gran disponibilidad de información sobre productos, precios y promociones en plataformas web de supermercados [1], [2]. Sin embargo, esta información se encuentra distribuida en múltiples sitios, lo que dificulta a los consumidores realizar comparaciones eficientes y tomar decisiones informadas [16], [18]. En ciudades como Barranquilla, esta problemática se intensifica debido a la ausencia de herramientas centralizadas que integren los datos de diferentes tiendas [2].

Desde la Ingeniería de Sistemas, este problema puede abordarse mediante el desarrollo de soluciones que automaticen la recolección, integración y presentación de información. En este contexto, las técnicas de Web Scraping permiten extraer datos desde múltiples fuentes web [1], [4], [5], mientras que los mecanismos de integración de datos facilitan su unificación en estructuras homogéneas [12], [13]. Adicionalmente, el uso de arquitecturas basadas en microservicios permite construir sistemas escalables, modulares y mantenibles [3], [9].

El presente proyecto propone el diseño e implementación de una aplicación de arquitectura modular escalable que permita comparar precios de productos en diferentes supermercados mediante la extracción automatizada de datos, su integración y posterior visualización en una interfaz interactiva. Esta solución busca facilitar la toma de decisiones de los usuarios, aportando mayor transparencia y eficiencia en el consumo.

II. DESCRIPCIÓN DEL PROBLEMA

El problema de investigación que se aborda es: ¿Cómo puede una plataforma web, apoyada en técnicas de Web Scraping y arquitecturas basadas en microservicios, centralizar y comparar de manera automatizada los precios de supermercados en Barranquilla, permitiendo además optimizar una lista de compras completa al identificar la tienda más conveniente para cada producto, facilitando la toma de decisiones informadas por parte de los consumidores?

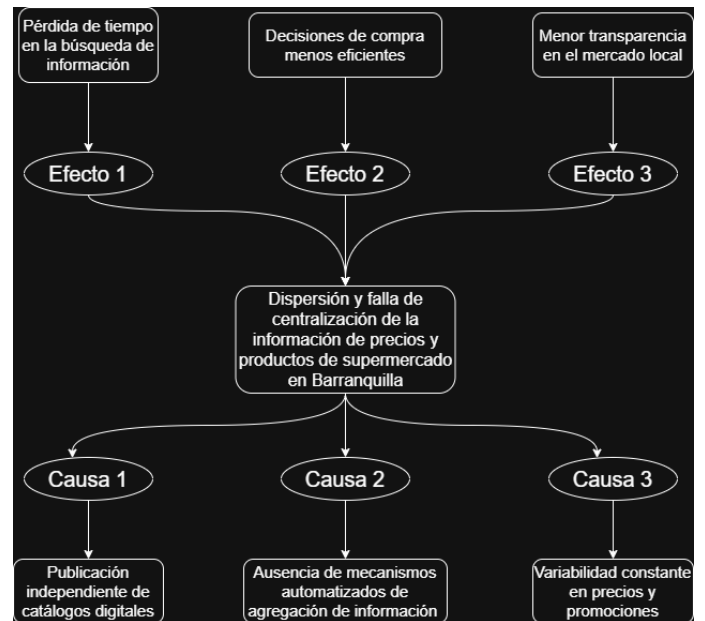


Fig. 1

Este problema refleja la necesidad de una solución computacional tangible que se materialice en un prototipo de aplicación web capaz de recolectar información desde múltiples fuentes, unificarla bajo un modelo estructurado y presentarla en una interfaz interactiva que permita realizar búsquedas, aplicar filtros y visualizar comparaciones actualizadas de productos. Para ilustrar las diferentes dimensiones de este desafío y su interrelación en el contexto del proyecto, se desarrollará un Árbol del Problema (ver Figura 1), el cual representará las causas asociadas a la dispersión de información, la falta de transparencia en precios y la ineficiencia en el proceso de comparación, así como sus consecuencias en el consumo local.

III. JUSTIFICACIÓN

El desarrollo de una plataforma de comparación de precios en supermercados resulta pertinente en un contexto donde la información se encuentra dispersa y es difícil de consultar de manera eficiente. Desde la perspectiva de la Ingeniería de Sistemas, este proyecto responde a la necesidad de diseñar soluciones que permitan integrar, procesar y visualizar grandes volúmenes de datos provenientes de múltiples fuentes digitales.

En términos de conveniencia, la propuesta permite a los usuarios ahorrar tiempo y dinero al facilitar el acceso centralizado a información de precios, evitando la consulta manual de múltiples plataformas. Esto se traduce en una mejora significativa en la toma de decisiones de compra.

Desde la relevancia social, el proyecto contribuye a una mayor transparencia en el mercado local, beneficiando directamente a los consumidores de Barranquilla al permitirles identificar las mejores opciones disponibles. Además, fomenta prácticas de consumo más informadas y eficientes.

En cuanto a las implicaciones prácticas, la solución aborda un problema real mediante la automatización de la recolección de datos utilizando técnicas de web scraping, las cuales han demostrado ser efectivas para la extracción de información web a gran escala [1]. Asimismo, la integración de estos datos en una plataforma interactiva mejora la accesibilidad y usabilidad de la

información.

Desde el valor teórico, el proyecto se apoya en estudios recientes sobre scraping, arquitecturas distribuidas y sistemas de comparación de precios, contribuyendo a la comprensión de cómo estas tecnologías pueden aplicarse en contextos locales de comercio minorista [2].

Finalmente, en términos de utilidad metodológica y tecnológica, la implementación de una arquitectura basada en microservicios permite validar enfoques modernos de desarrollo de software, caracterizados por la modularidad, escalabilidad y desacoplamiento de componentes [3]. Esto no solo fortalece el desarrollo del proyecto, sino que también aporta un caso práctico aplicable a futuros sistemas similares.

IV. OBJETIVOS

A. Objetivo General

Diseñar, desarrollar e implementar una aplicación para la comparación automatizada de precios de productos en supermercados, mediante técnicas de Web Scraping, integración de datos y una arquitectura modular, con el fin de centralizar la información y facilitar la toma de decisiones de los usuarios.

B. Objetivos Específicos

- Elaborar una revisión sistemática de la literatura relacionada con Web Scraping, integración de datos, arquitecturas modular y sistemas de comparación de precios.
- Diseñar la arquitectura de la solución definiendo los módulos de recolección de datos (crawler), procesamiento, optimización de lista de compras, almacenamiento y visualización.
- Implementar el módulo de Web Scraping para la extracción automatizada de información de productos y precios desde diferentes supermercados.
- Validar el funcionamiento del sistema evaluando la consistencia de los datos, tiempos de respuesta y correcta integración de los módulos.

V. METODOLOGÍA PROPUESTA

El desarrollo del proyecto se llevará a cabo mediante un enfoque incremental, estructurado en las siguientes etapas:

1. Revisión de la literatura

Se realizará una revisión sistemática de literatura (RSL) con el objetivo de identificar técnicas, herramientas y enfoques relacionados con Web Scraping, integración de datos y sistemas de comparación de precios.

2. Diseño de la arquitectura

Se definirá una arquitectura modular que contemple los siguientes componentes:

- Módulo de scraping (crawler)
- Módulo de procesamiento e integración de datos
- Base de datos para almacenamiento estructurado
- Backend para la gestión de consultas

- Frontend para la visualización de información

3. Implementación del módulo de scraping

Se desarrollarán scripts de extracción de datos utilizando herramientas como BeautifulSoup, Scrapy o Selenium, orientados a recolectar información de productos y precios desde sitios web de supermercados.

4. Desarrollo de la solución tecnológica

Se desarrollará una aplicación que permita:

- Buscar productos
- Visualizar precios en diferentes supermercados
- Comparar resultados de forma clara e intuitiva
- Procesar listas de compras ingresadas por el usuario y generar una recomendación por producto indicando la tienda con el menor precio disponible.

5. Validación del sistema

Se evaluará el sistema mediante:

- Pruebas de funcionamiento del scraping
- Verificación de consistencia de datos
- Verificación de la correcta asignación de tienda óptima por producto en listas de compras de prueba.
- Medición de tiempos de respuesta
- Pruebas básicas de usabilidad

VI. MARCO TEORICO

El autor Bhatt et al. en su trabajo titulado “Web scraping techniques and its applications: A review” [1], expone una revisión de técnicas de extracción de datos desde páginas web, destacando el uso de parsing de HTML, automatización mediante scripts y el empleo de herramientas especializadas como BeautifulSoup y Selenium. El estudio evidencia que estas técnicas permiten recolectar información estructurada desde múltiples fuentes digitales de forma automatizada, especialmente en entornos de comercio electrónico.

De manera complementaria, Bisht et al. en “Utilizing Python for Web Scraping and Incremental Data Extraction” [4], presentan una metodología orientada a la extracción incremental de datos, la cual permite actualizar información previamente recolectada sin necesidad de reprocesar completamente las fuentes. Este enfoque resulta relevante en contextos donde los datos, como los precios de productos, presentan variaciones frecuentes.

Asimismo, Sharma, en su trabajo “Web Crawling and Scraping: A Survey” [5], expone una diferenciación conceptual entre web crawling y web scraping, señalando que el crawling se enfoca en la exploración de páginas web mientras que el scraping se centra en la extracción de información específica. Esta distinción permite estructurar sistemas que integren ambos procesos de manera complementaria.

Por otra parte, Kahlon y Singh presentan una revisión sistemática en la que incorporan enfoques recientes basados en modelos de lenguaje para mejorar la extracción de datos,

destacando métricas de desempeño y desafíos asociados al tratamiento de información no estructurada [6].

A partir de lo expuesto, el presente trabajo identifica como conceptos clave:

- Extracción automatizada de datos desde múltiples sitios web
- Técnicas de parsing y automatización
- Procesos de crawling y scraping como componentes complementarios
- Extracción incremental de datos

Estos elementos constituyen la base para el diseño del módulo de recolección de información del sistema propuesto.

El autor Kazmali y Sayar, en su trabajo titulado “Web Scraping: Legal and Ethical Considerations...” [7], expone los principales desafíos legales asociados a la extracción automatizada de datos, destacando aspectos como el cumplimiento de términos de servicio, derechos de autor y uso adecuado de la información recolectada.

De manera complementaria, Pacheco Chaparro y Barrero Ramírez [8] analizan las implicaciones legales del web scraping en contextos actuales, enfatizando la necesidad de garantizar prácticas responsables en el uso de datos, especialmente cuando estos son utilizados en sistemas tecnológicos.

A partir de lo expuesto, el presente trabajo también identifica como conceptos clave:

- Cumplimiento de normativas y términos de uso
- Uso responsable de la información recolectada
- Consideraciones legales en la automatización de extracción de datos

Estos elementos son fundamentales para el desarrollo de una solución tecnológica viable y ética, y nos muestran la viabilidad legal en Colombia.

El autor Velepucha y Flores, en su trabajo titulado “A survey on microservices architecture: Principles, patterns and migration challenges” [3], expone los principios fundamentales de la arquitectura de modular por microservicios, destacando la descomposición de sistemas en servicios independientes, la escalabilidad y el desacoplamiento como elementos clave en el desarrollo de aplicaciones modernas.

De manera complementaria, Ataei y Staegemann [9] presentan la aplicación de patrones de microservicios en sistemas de datos, resaltando la importancia de la orquestación de servicios y la integración eficiente entre componentes distribuidos.

Asimismo, Gaol et al. [10] proponen patrones de diseño orientados a mejorar la mantenibilidad y flexibilidad en sistemas basados en microservicios, enfatizando la separación de responsabilidades y la modularidad.

Por otra parte, Insuasti et al. [11] presentan un diseño de arquitectura basado en microservicios aplicado a un sistema financiero, evidenciando mejoras en términos de escalabilidad, interoperabilidad y facilidad de evolución del sistema.

A partir de lo expuesto, el presente trabajo identifica como conceptos clave:

- Descomposición del sistema en servicios independientes
- Escalabilidad y mantenibilidad
- Integración y orquestación de servicios
- Aplicación de patrones de diseño en sistemas distribuidos

Estos elementos sustentan la arquitectura propuesta para la plataforma de comparación de precios.

El autor El Haddadi et al., en su trabajo titulado “Overview

on data ingestion and schema matching” [12], expone técnicas para la integración de datos provenientes de múltiples fuentes, destacando el proceso de schema matching como mecanismo para unificar estructuras de datos heterogéneas.

De manera complementaria, Rozony et al. [13] presentan una revisión de los desafíos asociados a la integración de datos en entornos de gran escala, señalando problemas como la heterogeneidad, inconsistencias y dificultades en la interoperabilidad entre sistemas.

Asimismo, Shao et al. [14] proponen métodos avanzados para la correspondencia de esquemas, facilitando la integración automática de datos provenientes de diferentes fuentes mediante técnicas de aprendizaje.

A partir de lo expuesto, el presente trabajo identifica como conceptos clave:

- Integración de datos provenientes de múltiples fuentes
- Normalización y unificación de estructuras de información
- Resolución de inconsistencias entre datos heterogéneos
- Procesos de ingestión y transformación de datos

Estos elementos permiten consolidar la información recolectada desde diferentes supermercados en una única plataforma.

El autor Singh et al., en su trabajo titulado “Price Comparison Using Web Scraping and Machine Learning” [2], expone un sistema que integra técnicas de web scraping y análisis de datos para comparar precios de productos, permitiendo mejorar la toma de decisiones de los usuarios mediante la visualización de información consolidada.

De manera complementaria, Dadiz [15] presenta un enfoque basado en analítica de datos para mejorar la comparación dinámica de precios en plataformas de comercio electrónico, destacando el uso de procesamiento de grandes volúmenes de información.

Asimismo, Nagaraj et al. [16] presentan el diseño de una plataforma web de comparación de precios que integra múltiples tecnologías de backend y bases de datos, evidenciando la viabilidad de este tipo de soluciones en entornos reales.

Por otra parte, Senbagavalli et al. [17] proponen un enfoque que combina técnicas de inteligencia artificial y web scraping para la predicción y comparación de precios, resaltando el potencial de estos sistemas en entornos dinámicos.

A partir de lo expuesto, el presente trabajo también identifica como conceptos clave:

- Comparación automatizada de precios
- Agregación de información desde múltiples fuentes
- Visualización de datos para la toma de decisiones
- Integración de técnicas de scraping con aplicaciones web

Estos trabajos nos muestran la viabilidad y necesidad de un proyecto como este, al ser algo que se ha intentado realizar en varias ocasiones.

VII. MARCO CONCEPTUAL

El desarrollo de plataformas de comparación de precios automatizadas se fundamenta en la integración de múltiples áreas de la Ingeniería de Sistemas, entre ellas el web scraping, las arquitecturas de software distribuidas y los sistemas de información orientados al comercio electrónico.

A. Web Scraping

El web scraping se define como el proceso automatizado de extracción de datos desde páginas web, permitiendo recolectar a partir de contenido no estructurado información estructurada [1]. Esta técnica es ampliamente utilizada en aplicaciones de análisis de datos, inteligencia de negocios y comercio electrónico. Según Sharma [5], el scraping y el crawling son componentes esenciales para la obtención de grandes volúmenes de información en tiempo real, lo que los convierte en herramientas clave para sistemas de comparación de precios.

Adicionalmente, estudios recientes destacan el uso de herramientas como Python, BeautifulSoup y Scrapy para realizar procesos de extracción incremental de datos, optimizando la eficiencia y reduciendo la redundancia en la información recolectada [4]. No obstante, el uso de estas técnicas también implica considerar aspectos legales y éticos relacionados con la privacidad y el uso de datos [7], [8].

B. Comparadores de precios en comercio electrónico

Los sistemas de comparación de precios permiten analizar productos provenientes de diferentes fuentes para facilitar la toma de decisiones del usuario. Investigaciones como la de Nagaraj et al. [16] demuestran que estos sistemas, combinados con scraping, permiten automatizar la recolección de datos de plataformas como Amazon o tiendas en línea, mejorando la eficiencia del proceso de búsqueda.

Por su parte, Singh et al. [2] plantean que la integración de técnicas de análisis de datos y aprendizaje automático en estos sistemas puede mejorar la precisión en la comparación y la visualización de información, aportando valor adicional al usuario. Asimismo, enfoques más recientes integran inteligencia artificial para predicción de precios y personalización de recomendaciones [17].

C. Arquitectura modular por microservicios

La arquitectura de microservicios se basa en la construcción de aplicaciones como un conjunto de servicios pequeños, independientes y desacoplados que se comunican mediante APIs [3]. Este enfoque permite mejorar la escalabilidad, mantenibilidad y flexibilidad de los sistemas modernos.

De acuerdo con Ataei y Staegemann [9], los microservicios son especialmente adecuados para sistemas que requieren procesamiento distribuido de datos y alta disponibilidad, como es el caso de plataformas que integran información de múltiples fuentes. Además, Gaol et al. [10] destacan que el uso de patrones de diseño en microservicios mejora la modificabilidad del software, facilitando la evolución del sistema a lo largo del tiempo.

D. Estado del arte

En los últimos años, se ha evidenciado un creciente interés en el desarrollo de plataformas que integran scraping, análisis de datos y arquitecturas distribuidas para aplicaciones de comercio electrónico. Diversos estudios han abordado la automatización de la comparación de precios, incorporando tecnologías como machine learning, sistemas de recomendación y procesamiento en tiempo real [2], [17].

Sin embargo, la mayoría de estas soluciones se enfocan en grandes plataformas globales, existiendo una brecha en aplicaciones orientadas a contextos locales, como ciudades específicas.

Asimismo, muchas de estas soluciones presentan limitaciones en cuanto a la transparencia de los datos recolectados y la frecuencia de actualización de precios, lo que puede afectar la confiabilidad de la información presentada al usuario.

En este sentido, el presente proyecto aporta al estado del arte al proponer una solución adaptada al contexto de Barranquilla, integrando tecnologías modernas y abordando problemáticas reales de los consumidores locales.

VIII. ARQUITECTURA LÓGICA

La arquitectura lógica del sistema se fundamenta en un enfoque basado en microservicios, el cual permite integrar múltiples fuentes de datos dentro de una plataforma de comparación de precios de supermercados. Este enfoque facilita la organización del sistema en componentes independientes, cada uno encargado de una funcionalidad específica, lo que mejora la escalabilidad, mantenibilidad y evolución del software.

El sistema inicia con la obtención de información desde fuentes externas correspondientes a supermercados locales, específicamente Éxito, Olímpica y D1. Estas fuentes son accedidas mediante técnicas de web scraping, las cuales permiten extraer datos relevantes como el nombre del producto, su precio, disponibilidad y tienda de origen directamente desde sus plataformas digitales. Este proceso automatizado permite superar la dispersión de la información, centralizándola en una única plataforma accesible para el usuario.

Una vez recolectados los datos, el sistema realiza un proceso de transformación y normalización con el objetivo de unificar productos equivalentes provenientes de diferentes supermercados. Este proceso incluye la limpieza de datos, la homologación de nombres y la estructuración de la información en un modelo consistente. Posteriormente, los datos son almacenados en bases de datos relacionales, lo que permite su consulta eficiente y su reutilización en futuras solicitudes.

El flujo de operación continúa con la interacción del usuario a través del frontend, donde se realizan búsquedas y consultas. Estas solicitudes son gestionadas por un componente orquestador en el backend, el cual coordina la consulta de datos previamente almacenados y, en caso necesario, activa el módulo de crawling para obtener información actualizada desde las fuentes externas. Este enfoque permite combinar datos en tiempo real con información persistente, optimizando tanto el rendimiento como la actualización de la información.

Adicionalmente, el sistema incorpora un módulo de optimización de lista de compras. El usuario ingresa una lista de productos deseados, la cual es procesada por el backend para consultar los precios de cada ítem en todas las tiendas disponibles. Mediante un proceso de comparación automatizada, el sistema determina la tienda con el precio más conveniente para cada producto de forma individual, presentando al usuario un resultado estructurado que indica, por cada ítem, el establecimiento recomendado para su compra [2], [17]. Este módulo se apoya en los datos previamente almacenados y, cuando es necesario, activa el crawler para

obtener precios actualizados.

Finalmente, los resultados obtenidos son consolidados mediante un proceso de agregación que unifica la información de los distintos supermercados, permitiendo presentar al usuario una respuesta clara, organizada y comparable. Este proceso es fundamental para reducir la complejidad en la toma de decisiones y mejorar la experiencia del usuario al momento de comparar precios entre diferentes tiendas.

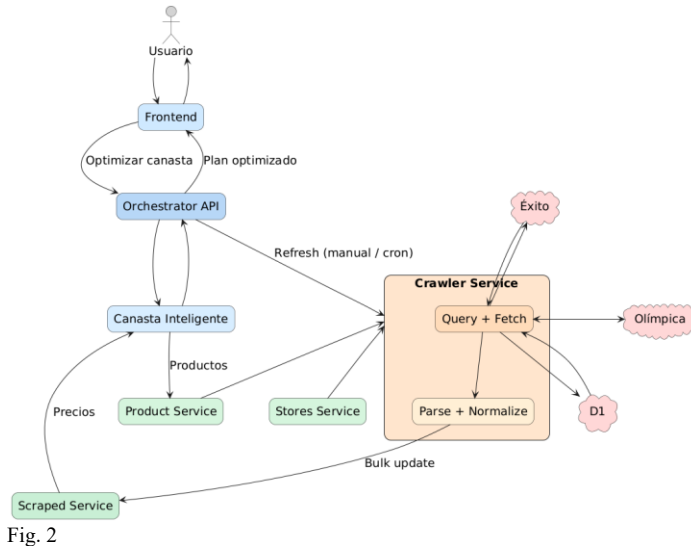


Fig. 2

IX. ARQUITECTURA FÍSICA

Desde la perspectiva de la arquitectura física, el sistema se implementa como una solución distribuida que permite el despliegue independiente de sus componentes en una infraestructura escalable. El usuario accede a la plataforma a través de un navegador web, el cual consume una aplicación frontend desplegada en una red de distribución de contenido.

Las solicitudes generadas son dirigidas a un API Gateway, que actúa como punto de entrada único y se encarga de enrutar las peticiones hacia los diferentes servicios del sistema.

El backend está compuesto por múltiples microservicios desplegados de forma independiente, incluyendo los servicios de usuarios, productos, tiendas, el módulo de crawling y el módulo de optimización de lista de compras. Estos dos últimos tienen un papel fundamental: el primero se encarga de realizar el web scraping de los sitios web de Éxito, Olímpica y D1, recolectando información actualizada de los productos disponibles en cada uno de estos supermercados; el segundo, por otro lado, recibe una lista de productos ingresada por el usuario, consulta los precios disponibles en cada tienda a través de los datos almacenados en las bases de datos PostgreSQL y, en caso de requerirse información actualizada, activa el servicio de crawling. Con base en los resultados obtenidos, el módulo retorna una recomendación individualizada por producto, indicando la tienda con el precio más conveniente para cada ítem.

Para mejorar el rendimiento del sistema, el proceso de recolección de datos puede ejecutarse de manera asíncrona mediante el uso de mecanismos de mensajería, lo que permite desacoplar el scraping del flujo principal de consultas. Esto contribuye a reducir la latencia en las respuestas al usuario y a mejorar la capacidad del sistema para manejar múltiples

solicitudes simultáneamente.

Los datos recolectados son almacenados en bases de datos PostgreSQL, donde son organizados y optimizados para consultas rápidas. Cada microservicio gestiona su propio acceso a los datos, lo que garantiza la independencia de los componentes y permite escalar cada servicio de manera individual según la demanda.

En conjunto, la arquitectura física propuesta permite soportar un sistema robusto, capaz de integrar información de múltiples supermercados en tiempo real, garantizando alta disponibilidad, eficiencia en el procesamiento de datos y una experiencia de usuario fluida.

Fig. 3

X. PROTOTIPO

1. Descripción general del prototipo

Se desarrolló un prototipo web funcional denominado "Luvao", cuyo objetivo es centralizar información de precios de productos obtenida desde diferentes supermercados y presentar comparaciones que faciliten la toma de decisiones de compra.

El sistema integra módulos de extracción automática de datos, almacenamiento, procesamiento y visualización, permitiendo consultar productos, comparar precios y optimizar listas de compras.

Adicionalmente, la sección especializada para visualizar los datos obtenidos de todas las tiendas permite un análisis económico de como varían los precios, qué tiendas son las más económicas, y también qué tiendas no tienen siquiera todos los productos de la canasta básica alimentaria, según el DANE.

2. Tecnologías utilizadas

Componente	Tecnología
Frontend	React
Backend	FastAPI / Spring Boot / Node.js
Base de datos	PostgreSQL
Web Scraping	FastAPI + Uvicorn + BeautifulSoup4 + Selenium + httpx
Contenedores	Docker
API Gateway	Orquestador NestJS ('/api/orchestrator')
Control de versiones	GitHub
Documentación	Swagger

3. Módulo de recolección de datos (Web Scraping)

El módulo de recolección de datos constituye uno de los componentes fundamentales de la plataforma, ya que permite obtener información actualizada de productos directamente desde los catálogos públicos de supermercados. Su función principal es automatizar la consulta de fuentes externas, extraer los datos relevantes de cada producto y enviarlos al servicio de almacenamiento e integración para su posterior consulta desde

la interfaz de usuario.

En la implementación actual del proyecto, el crawler está desarrollado en FastAPI y se apoya en peticiones HTTP asincrónicas mediante httpx para consultar los catálogos públicos de supermercados. El proceso se ejecuta desde el servicio de orquestación cuando el usuario solicita una actualización manual de la información, lo que permite refrescar los datos sin necesidad de reiniciar el sistema completo. Esta estrategia responde a un enfoque de actualización bajo demanda, adecuado para un prototipo funcional en el que los precios pueden variar con frecuencia.

El crawler consulta actualmente tres supermercados integrados en la solución: Éxito, D1 y Olímpica. La selección de estas fuentes responde a su relevancia dentro del contexto local y a la disponibilidad de endpoints públicos que facilitan la extracción automatizada. La arquitectura del sistema está preparada para incorporar nuevas cadenas de supermercados en etapas posteriores, ampliando así el alcance de la comparación de precios.

Durante el proceso de extracción, el crawler recorre las consultas de productos definidas en el sistema y realiza solicitudes sobre los catálogos de cada supermercado. Para cada respuesta recibida, identifica los productos asociados y obtiene la información disponible en el catálogo. En esta etapa, los datos que efectivamente se extraen en la implementación actual son principalmente el nombre del producto, su precio, su URL y sus calorías. Esto es usado para el análisis de datos y el cálculo de estadísticas útiles para el usuario.

La información recolectada es enviada al servicio de productos *scrapeados*, donde se almacena de forma estructurada para su posterior consulta, filtrado y visualización. Este flujo permite que los datos extraídos desde diferentes supermercados se normalicen bajo un mismo esquema, facilitando su comparación en la interfaz principal del sistema.

Adicionalmente, el proceso de scraping genera mensajes de ejecución y trazas de control que permiten monitorear el estado de la recolección. Estos logs reportan, por ejemplo, el supermercado consultado, la query procesada, los intentos de reintento ante errores y la cantidad de productos obtenidos. Dichos mensajes son útiles tanto para depuración como para verificación de funcionamiento, ya que permiten identificar posibles fallos de conexión o inconsistencias en las respuestas de los sitios externos.

```
"generated_at": "2026-05-05 21:53:55",
"technical": {
  "checks": {
    "stores_available": {
      "pass": true,
      "message": "Stores disponibles: 3"
    },
    "products_available": {
      "pass": true,
      "message": "Productos canónicos disponibles: 2"
    },
    "compare_has_results": {
      "pass": true,
      "message": "Comparación arroz devolvió 30 candidatos"
    },
    "optimize_consistency": {
      "pass": true,
      "message": "Total estimado 10090 coincide con suma de subtotales"
    },
    "d1_persisted_data": {
      "pass": true,
      "message": "Productos persistidos para D1: 27"
    },
    "job_integration_success": {
      "pass": true,
      "message": "Job terminó en estado success"
    }
  }
}
```

Captura 1.

4. Base de datos e integración de información

4.1 Resumen y propósito

El subsistema de persistencia centraliza los datos estructurados que utiliza la plataforma para comparar productos entre supermercados. Su finalidad es: (i) almacenar la lista canónica de productos a monitorear, (ii) guardar los resultados del scraping por tienda y consulta, y (iii) permitir consultas eficientes y filtros para la interfaz y los procesos de optimización de listas de compra.

4.2 Modelo entidad-relación (alto nivel)

Entidades principales:

- *User*: usuarios del sistema (registro y autenticación).
- *Product*: lista canónica de productos a monitorear (únicos por nombre).
- *Store*: metadatos de tiendas / scrapers (nombre, baseUrl, searchPath).
- *ScrapedProduct*: resultados del scraping (un registro por producto obtenido por tienda y query en una ejecución de scraping).

Relación y cardinalidad (conceptual):

- Un Store produce muchos ScrapedProduct (1:N).
- Una Product (nombre canónico) puede corresponder a muchos ScrapedProduct (1:N).

4.3 Proceso de homologación (cómo se normalizan y asocian productos)

El sistema implementa un proceso pragmático y orientado al prototipo para asociar (homologar) los nombres extraídos del scraping con la lista canónica de productos:

1. Lista canónica (fuente de consultas):

Product almacena los términos (nombres) que el sistema desea rastrear. Estas cadenas se usan como "queries" de búsqueda por el crawler.

2. Extracción por scrapers:

Para cada query de *Product* el crawler consulta los endpoints públicos de la tienda (ej.: las API de búsqueda de Éxito y Olímpica) y obtiene una lista de items. Los scrapers aplican limpieza básica (por ejemplo strip() sobre productName) y extraen name y price cuando están disponibles.

3. Filtrado/correspondencia query → producto scrapeado:

Implementación actual: para cada query se filtran los productos obtenidos comprobando si la query aparece como substring (comparación insensible a mayúsculas) dentro del name extraído. Esta estrategia simple permite mapear variantes de nombre siempre que la query esté presente en el texto del producto (if query.lower() in p["name"].lower() en el crawler).

Justificación: es una heurística robusta para listas canónicas cortas y evita la complejidad de reconciliación con algoritmos de fuzzy matching en la primera etapa del prototipo.

4. Normalización mínima aplicada:

Limpieza aplicada por los scrapers: strip() y eliminación de entradas vacías; los precios se convierten a números cuando el endpoint los entrega. En app.py del crawler, cuando no hay url disponible se guarda null, y la disponibilidad se infiere (por ejemplo, availability = price is not None and price > 0).

Observación: el modelo de datos y las APIs están diseñadas para admitir atributos adicionales (p. ej. category, url) pero la extracción actual no captura categoría ni URL de forma generalizada; el esquema soporta esos campos para ampliación futura.

5. Persistencia y actualización atómica

Para cada combinación storeName + query el servicio de scraped-service elimina los registros antiguos de esa combinación y crea los nuevos en una transacción atómica. Esto garantiza que las consultas posteriores devuelvan un conjunto consistente para esa tienda y query (evita mezclas parciales de ejecuciones antiguas y nuevas).

5. Interfaz de usuario

La interfaz, con un diseño moderno y de fácil uso para el usuario, se separa en dos ventanas principales. La ventana de búsqueda de productos y la ventana de visualización de datos.

5.1. Búsqueda de productos



Captura 2.

5.2. Visualización de datos



Captura 3.

6. Flujo de funcionamiento del sistema



Fig. 4

XI. RESULTADOS

Se implementó una plataforma web funcional capaz de extraer información de productos desde diferentes supermercados, almacenar los datos recolectados, procesarlos y presentarlos mediante una interfaz de consulta y comparación de precios. La integración de los módulos de scraping, procesamiento, almacenamiento y visualización permitió validar la viabilidad técnica de la solución propuesta.

1. Resultados de la extracción de datos

Se realizó la validación funcional del prototipo utilizando una lista de compras de referencia para un hogar de cuatro personas durante un período de un mes, considerando un requerimiento promedio de 2200 kcal por persona al día.

Los resultados obtenidos muestran que el sistema logró una **cobertura del 100 %**, encontrando coincidencias para los **39 productos** incluidos en la lista de prueba. Esto indica que la base de datos integrada permitió identificar alternativas disponibles para todos los productos requeridos.

El optimizador generó un **costo total estimado de \$1.577.567 COP** para la canasta completa. Asimismo, calculó un **costo promedio diario por persona de \$13.146 COP**, equivalente a **\$5.557 COP por cada 1000 kcal consumidas**, permitiendo analizar la eficiencia económica de la alimentación propuesta.

En términos nutricionales, la solución alcanzó un **cumplimiento calórico del 107,5 %**, obteniendo **283.883 kcal planificadas** frente al objetivo teórico definido para el escenario evaluado. Esto evidencia que la selección de productos satisface adecuadamente los requerimientos energéticos establecidos.

La distribución del gasto entre supermercados mostró una participación relativamente equilibrada:

- Olímpica: **\$564.375 COP (35,8 %)**
- Éxito: **\$530.502 COP (33,6 %)**
- D1: **\$482.030 COP (30,6 %)**

Estos resultados indican que la estrategia óptima no concentra todas las compras en una sola tienda, sino que aprovecha los precios más convenientes disponibles en cada supermercado.

También es importante recalcar que la diferencia de precio entre D1 y las otras tiendas no se debe tanto a unos precios menores, los cuales sí poseen, sino a que D1 no tiene a la venta la canasta básica completa, por lo que hay productos que no suman en su total.

Por categorías de alimentos, el sistema logró cubrir completamente los productos requeridos en cereales y harinas, tubérculos y plátanos, legumbres, verduras y hortalizas, frutas, proteínas, lácteos, grasas y complementos, endulzantes y básicos, así como bebidas y otros, alcanzando una cobertura del **100 % en todas las categorías evaluadas**.

Finalmente, el análisis comparativo de precios identificó un **ahorro potencial acumulado de \$2.803.268 COP** a partir de **39 registros comparables entre supermercados**, demostrando la capacidad de la plataforma para detectar oportunidades de ahorro mediante la selección automática de la mejor alternativa disponible para cada producto.

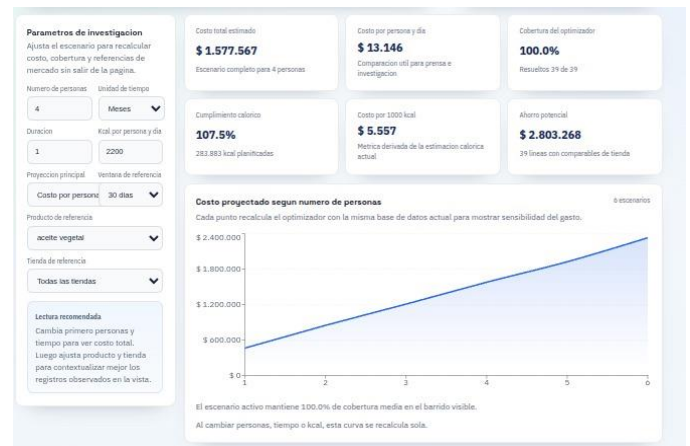


Fig. 5

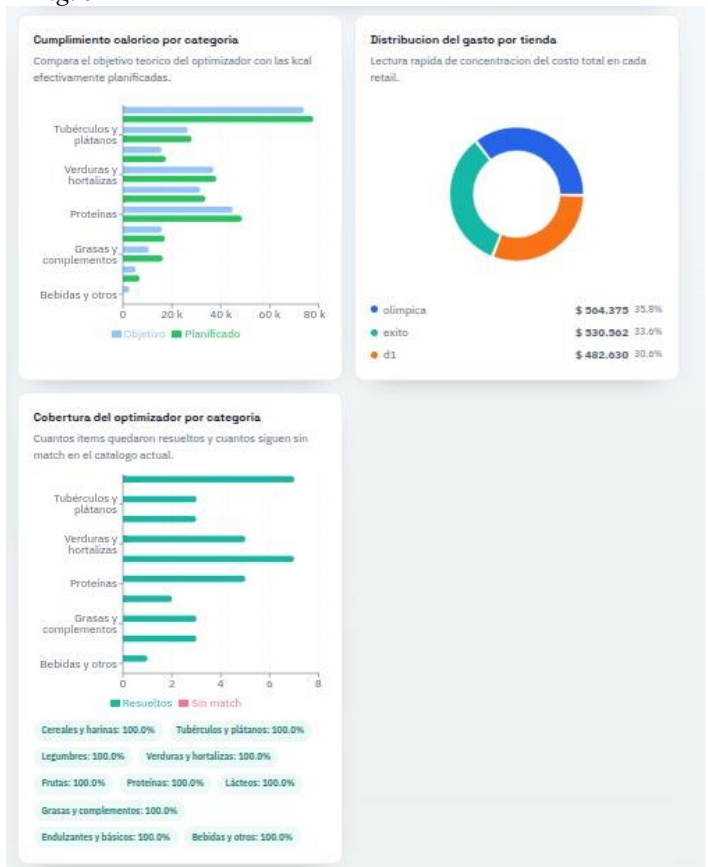


Fig. 6

XII. EVALUACIÓN CRUZADA

Evaluación de nuestro proyecto realizada por el grupo realizador del proyecto “Plataforma Web y Movil de Colaboración Ciudadana para la Gestión Preventiva y Detección de Puntos Críticos de Seguridad Vial en Barranquilla”:

Características	Definición o descripción	1..5
Understandability	¿Fácil de comprender?	5
Documentation	¿Documentación de usuario completa, apropiada y bien estructurada?	4
Buildability	¿Fácil de construir en un sistema compatible? (Close-Open)	5

Installability	¿Fácil de instalar en un sistema compatible?	5
Learnability	¿Fácil de aprender a usar sus funciones?	5
Identity	¿La identidad del proyecto / software es clara y única?	4.8
Copyright	¿Es fácil ver quién posee el proyecto / software?	4.7
Licensing	¿Adopción de la licencia apropiada?	5
Governance	¿Fácil de entender cómo se ejecuta el proyecto y cómo se gestiona el desarrollo del software?	5
Community	¿Evidencia de comunidad actual / futura?	4
Accessibility	¿Evidencia de capacidad de descarga actual / futura?	5
Testability	¿Fácil de probar la corrección de las funciones caja negra?	5
Portability	¿Utilizable en múltiples plataformas?	5
Supportability	¿Evidencia de soporte para desarrolladores actuales / futuros?	5
Analysability	¿Fácil de entender a nivel fuente?	5
Changeability	¿Fácil de modificar y aportar cambios a los desarrolladores?	5
Evolvability	¿Evidencia de desarrollo actual / futuro?	5
Interoperability	¿Interoperable con otro software requerido / relaciona	5
Modelo de evaluación basado en el estándar ISO 9126 ISO 15504 + ESCALA DE LIKERT		

XIII. CONCLUSIONES

El desarrollo de la plataforma permitió demostrar la viabilidad de integrar técnicas de Web Scraping, procesamiento de datos y arquitecturas basadas en microservicios para construir una herramienta de comparación automatizada de precios en supermercados. La solución implementada logró centralizar información proveniente de múltiples fuentes, reduciendo la dispersión de los datos y facilitando su consulta desde una única interfaz web.

Los resultados obtenidos evidencian que el sistema cumple con los objetivos planteados, al permitir la extracción automática de información desde los supermercados Éxito, Olímpica y D1, su almacenamiento estructurado y la generación de recomendaciones de compra basadas en criterios de costo.

Asimismo, el módulo de optimización de listas de compra confirmó la utilidad práctica de la propuesta, ya que fue capaz de identificar automáticamente la alternativa más conveniente para cada producto y estimar indicadores económicos relevantes, tales como costo total, costo diario por persona, costo por 1000 kcal y ahorro potencial. Esto permite transformar grandes volúmenes de información dispersa en recomendaciones comprensibles y útiles para el consumidor final.

Desde el punto de vista arquitectónico, el uso de una estructura modular basada en microservicios facilitó la separación de responsabilidades entre los distintos componentes del sistema, permitiendo un desarrollo más organizado, mantenible y escalable. Esta característica resulta especialmente importante en aplicaciones que dependen de múltiples fuentes externas y que requieren incorporar nuevas funcionalidades o supermercados de forma progresiva.

Finalmente, se identifican oportunidades de trabajo futuro orientadas a ampliar el número de supermercados integrados, mejorar los mecanismos de homologación de productos

mediante técnicas de procesamiento de lenguaje natural o aprendizaje automático, incorporar actualizaciones automáticas periódicas de precios y desarrollar modelos de recomendación más avanzados que consideren preferencias del usuario, ubicación geográfica y patrones históricos de consumo. Estas mejoras permitirían incrementar la precisión de las recomendaciones y ampliar el impacto de la plataforma dentro del contexto del comercio minorista local.

REFERENCIAS

- [1] C. Bhatt, A. Bisht, R. Chauhan, A. Vishvakarma, M. Kumar and S. Sharma, "Web scraping techniques and its applications: A review," in 2023 3rd International Conference on Innovative Sustainable Computational Technologies (CISCT), 2023, pp. 1–8.
- [2] N. Singh, A. Rana and A. Chaudhary, "Price Comparison Using Web Scraping and Machine Learning," in 2023 International Conference on Computer Science and Emerging Technologies (CSET), 2023, pp. 1–5.
- [3] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," IEEE Access, vol. 11, pp. 88339–88358, 2023.
- [4] V. Bisht, R. Choyal, A. S. Negi and E. K. Singh, "Utilizing Python for Web Scraping and Incremental Data Extraction," in 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), 2023, pp. 1450–1455.
- [5] G. Sharma, "Web Crawling and Scraping: A Survey," in 2024 International Conference on Healthcare Innovations, Software and Engineering Technologies (HISSET), 2024, pp. 190–192.
- [6] N. Kahlon and W. Singh, "A Systematic Review of Web Scraping: Techniques, LLM-Enhanced Approaches, Performance Metrics, and Legal-Ethical Issues."
- [7] A. S. Kazmali and A. Sayar, "Web Scraping: Legal and Ethical Considerations in General and Local Context-A Review," Procedia Computer Science, vol. 259, pp. 1563–1572, 2025.
- [8] J. M. Pacheco Chaparro and L. Barrero Ramírez, "Implicaciones legales del web scraping en el entrenamiento de modelos de inteligencia artificial generativa," Revista La Propiedad Inmaterial, no. 38, pp. 167–189, 2024.
- [9] P. Ataei and D. Staegemann, "Application of microservices patterns to big data systems," Journal of Big Data, vol. 10, no. 1, p. 56, 2023.
- [10] F. L. Gaol, A. N. Fajar, A. S. Girsang and T. Matsuo, "Microservices Design Pattern in Action: Improving Modifiability in Microservices-Based Software Development," IEEE Access, 2025.
- [11] E. Insuasti, B. Cisneros and P. Morillo, "A Software Architecture Design Based on Microservices for an E-wallet in Ecuador," in Proc. 2024 7th International Conference on Computers in Management and Business (ICCMB), 2024, pp. 185–192.

- [12] O. El Haddadi et al., "Overview on data ingestion and schema matching," *Data and Metadata*, vol. 3, p. 219, 2024.
- [13] F. Z. Rozony, M. N. A. Aktar, M. Ashrafuzzaman and A. Islam, "A systematic review of big data integration challenges and solutions for heterogeneous data sources," *Academic Journal on Business Administration, Innovation & Sustainability*, vol. 4, no. 4, pp. 1–18, 2024.
- [14] L. Shao, X. Hao and J. T. Zhou "A Composite Schema Matching Method Based on Contrastive Learning in Data Integration," in *2025 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2025, pp. 2109–2114.
- [15] B. Dadiz, "Design and Implementation of Big Data and Analytics to Enhance Dynamic Price Comparison Analytics," 2024.
- [16] P. Nagaraj, V. Muneeswaran, A. V. S. R. Pavan Naidu, N. Shanmukh, P. V. Kumar and G. S. Satyanarayana, "Automated E-Commerce Price Comparison Website using PHP, XAMPP, MongoDB, Django, and Web Scrapping," in *2023 International Conference on Computer Communication and Informatics (ICCCI)*, 2023, pp. 1–6.
- [17] M. Senbagavalli, N. Kumar, A. Sungheetha, D. Kumar and S. Ghosh, "IntelliPrice: A Unified Review of AI and Web Scraping Techniques for Dynamic E-Commerce Price Prediction and Comparison," in *2025 IEEE India Council International Subsections Conference (INDISCON)*, 2025, pp. 1–8.
- [18] A. Shaikh, R. Khan, K. Panokher, M. K. Ranjan and V. Sonaje, "E-commerce Price Comparison Website Using Web Scraping," *International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences*, vol. 11, no. 3, May–Jun. 2023, doi: 10.37082/ijirms.v11.i3.230223.
- [19] Departamento Administrativo Nacional de Estadística (DANE), "Índice de Precios al Consumidor (IPC)," *dane.gov.co*. [En línea]. Disponible en: <https://www.dane.gov.co/index.php/estadisticas-por-tema/precios-y-costos/indice-de-precios-al-consumidor-ipc>. [Accedido: 22-may-2026].